Лабораторная работа № 2

Тема: Хеширование и безопасное хранение паролей

Цель: Исследовать работу Argon2 и проверить безопасное хранение паролей в существующем REST API, самостоятельно реализовать смену пароля.

Дисциплина: РО 4.4. Аутентификация, авторизация, безопасность и интеграция frontend/backend

> **Важно**
>
> Продолжайте проект из ЛР-1. User, таблица users, argon2, passwordService.js, authService.js и маршруты /api/auth/register, /api/auth/login уже существуют. Не создавайте их повторно. Новые миграции для этой работы не требуются.

1. Подготовка

1. Откройте проект express-shop (или проект, в котором выполнялась ЛР-1).
2. Проверьте наличие пакета argon2 в package.json и файла services/passwordService.js.
3. Проверьте, что регистрация и вход из ЛР-1 работают.
4. Для дальнейших проверок используйте учебные пароли, а не настоящие.

Если пакет отсутствует, установите его:

```
npm install argon2
```

2. Эксперимент: один пароль — два разных хеша

Создайте новый файл argon2-test.js в корне проекта. В существующие сервисы изменений пока не вносите.

```
import * as argon2 from "argon2";

const password = "Example123!";

const hash1 = await argon2.hash(password);
const hash2 = await argon2.hash(password);

console.log("Первый хеш:", hash1);
console.log("Второй хеш:", hash2);
console.log("Хеши равны:", hash1 === hash2);

console.log("Верный пароль:",
  await argon2.verify(hash1, password));
console.log("Неверный пароль:",
  await argon2.verify(hash1, "WrongPassword!"));
```

Запустите файл:

```
node argon2-test.js
```

Проверка	Ожидаемый результат
hash1 === hash2	false
verify(hash1, правильный пароль)	true
verify(hash1, неправильный пароль)	false

Пояснение: библиотека создаёт случайный salt при каждом вызове hash(). Поэтому хеши одного пароля отличаются. verify() умеет проверить исходный пароль по сохранённому хешу.

3. Самостоятельный эксперимент: поменяйте пароль

5. В файле argon2-test.js замените строку password на другой учебный пароль.
6. Снова запустите программу и зафиксируйте три результата.
7. Объясните, почему нельзя проверять пароль выражением argon2.hash(password) === savedHash.

4. Проверка существующего сервиса

Откройте services/passwordService.js из ЛР-1. Он должен содержать функции hashPassword() и verifyPassword(), использующие argon2.hash() и argon2.verify(). Повторно создавать файл не нужно.

```
export async function hashPassword(password) {
  return argon2.hash(password);
}

export async function verifyPassword(hash, password) {
  return argon2.verify(hash, password);
}
```

Найдите в services/authService.js места вызова обеих функций. Запишите: при какой операции используется hashPassword(), а при какой — verifyPassword().

5. Проверьте данные в PostgreSQL

Выполните регистрацию нового учебного пользователя через уже работающий POST /api/auth/register. Используйте новый email, например lab2@example.com, и пароль Example123! (только учебный пример).

В pgAdmin откройте Query Tool и выполните простой запрос:

```
SELECT id, name, email, password_hash, role
FROM users
WHERE email = 'lab2@example.com';
```

8. Найдите поле password_hash: в нём должна быть длинная строка хеша, обычно начинающаяся с $argon2id$.
9. Убедитесь, что открытого пароля Example123! нет в таблице.
10. Не копируйте в отчёт реальные пароли или полные хеши. Достаточно скриншота структуры результата с замаскированным значением.

6. Проверка входа через существующее API

Запрос	Данные	Результат
POST /api/auth/login	lab2@example.com + Example123!	Успешный вход
POST /api/auth/login	lab2@example.com + WrongPassword!	HTTP 401
POST /api/auth/login	unknown@example.com + Example123!	HTTP 401

Убедитесь, что сообщения при неправильном пароле и неизвестном email одинаковые, а passwordHash не возвращается клиенту. Код маршрутов и сервисов из ЛР-1 без необходимости не переписывайте.

7. Самостоятельное задание: смена пароля пользователя

Добавьте в учебный API новую операцию POST /api/auth/change-password. Она должна принимать email, oldPassword и newPassword. Это отдельная учебная операция, позволяющая применить hashPassword() и verifyPassword() по новому сценарию.

> **Важно**
>
> Эта операция не заменяет полноценную аутентифицированную смену пароля. В курсовой с JWT или сессиями идентификатор пользователя нужно брать из проверенной аутентификации, а не доверять одному email из тела запроса.

Последовательность действий в Service:

11. Найти пользователя по email через prisma.user.findUnique().
12. Проверить oldPassword по сохранённому passwordHash через verifyPassword().
13. Если пользователя нет или старый пароль неверен — вернуть одинаковую ошибку, не изменяя базу.
14. Проверить, что newPassword — непустая строка длиной не менее 8 символов.
15. Получить новый хеш через hashPassword(newPassword).
16. Обновить только passwordHash пользователя через prisma.user.update().
17. Вернуть сообщение об успехе без passwordHash.

Каркас для самостоятельного заполнения (services/authService.js):

```
export async function changePassword(email, oldPassword, newPassword) {
  // 1. Найдите пользователя по email.
  // 2. Проверьте старый пароль через verifyPassword().
  // 3. Проверьте новый пароль.
  // 4. Создайте новый хеш через hashPassword().
  // 5. Обновите passwordHash через prisma.user.update().
  // 6. Верните безопасный результат.
}
```

В routes/authRoutes.js и controllers/authController.js самостоятельно подключите POST /change-password. Используйте существующее разделение Route → Controller → Service.

Пример тела запроса:

```
{
  "email": "lab2@example.com",
  "oldPassword": "Example123!",
  "newPassword": "NewExample123!"
}
```

8. Проверка самостоятельного задания

Сценарий	Ожидаемый результат
Правильный oldPassword	Новый пароль сохраняется, сообщение об успехе
Неправильный oldPassword	Ошибка; пароль не меняется
Слишком короткий newPassword	HTTP 400; пароль не меняется
Login со старым паролем после изменения	HTTP 401
Login с новым паролем после изменения	Успешный вход

После смены пароля проверьте через pgAdmin, что password_hash изменился. Старый и новый хеши не нужно включать в ответ API.

9. Требования к отчёту

18. Скриншот запуска argon2-test.js с результатами false, true и false.
19. Краткое объяснение назначения salt и причины различия хешей.
20. Скриншот записи пользователя в pgAdmin без раскрытия полного хеша.
21. Результаты проверки успешного и ошибочного входа.
22. Для самостоятельного задания: код новой операции и результаты пяти проверок.
23. Ответ на вопрос, почему passwordHash нельзя возвращать в HTTP-ответе.

10. Контрольные вопросы

24. Почему хеш одного и того же пароля отличается при повторном вызове hash()?
25. Чем hash() отличается от verify()?
26. Почему нельзя сравнивать два заново созданных хеша с помощью ===?
27. Как называется поле для хранения хеша в PostgreSQL?
28. Почему при смене пароля нужно сначала проверить старый пароль?
29. Какие данные нельзя возвращать в API после регистрации или входа?